We're going to show you both techniques. We'll start with the approach we generally recommend—the matcher DSL—and then show you what the same matcher would look like as a Ruby class.

A Minimal Custom Matcher

To define a matcher using the DSL, we call RSpec::Matchers.define, passing the matcher name and a block containing the matcher definition:

12-creating-custom-matchers/06/custom_matcher/spec/support/matchers.rb

```
RSpec::Matchers.define :have_a_balance_of do |amount|
  match { |account| account.current_balance == amount }
end
```

The outer block receives any arguments passed to the matcher. When a spec calls have_a_balance_of(amount), RSpec will pass the amount into this block.

The match method defines the actual match/no-match logic. The inner block receives the *subject* of the expectation (the account), and returns a truthy value if the account balance matches the expected amount.

Improving the Custom Matcher Failure Messages

This matcher was easy to write, but we shouldn't declare victory just yet. Here's the output that it produces when a spec fails:

```
1) `have_a_balance_of(amount)` fails when the balance does not match
   Failure/Error: expect(account).to have_a_balance_of(35)
     expected #<Account name="Checking"> to have a balance of 35
   # ./spec/initial_account_spec.rb:17:in `block (2 levels) in <top
```

The failure message tells us that the account *should* have had a balance of 35. But it doesn't say what the *actual* balance was. We can add this information using the failure_message and failure_message_when_negated methods:

12-creating-custom-matchers/07/custom_matcher/spec/support/matchers.rb

```
RSpec::Matchers.define :have_a_balance_of do |amount|
  match { |account| account.current_balance == amount }
➤  failure_message { |account| super() + failure_reason(account) }
➤  failure_message_when_negated { |account| super() + failure_reason(account) }

private

➤  def failure_reason(account)
➤    ", but had a balance of #{account.current_balance}"
➤  end
end
```

We define two methods so that RSpec can supply our custom failure text both for expect(...).to(...) and for expect(...).not_to(...). As with match, these methods each